My Project

Generated by Doxygen 1.17.0

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

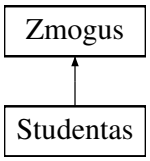
Here is a list of all documented files with brief descriptions:

Chapter 4

Class Documentation

4.1 Studentas Class Reference

Inheritance diagram for Studentas:



Public Member Functions

- **Studentas** (const std::string &vardas, const std::string &pavarde, const std::vector< int > &paz, int egz)
- **Studentas** (const Studentas &other)
- **Studentas** (Studentas &&other) noexcept
- Studentas & **operator=** (const Studentas &other)
- Studentas & **operator=** (Studentas &&other) noexcept
- bool **operator<** (const Studentas &other) const
- bool **operator>** (const Studentas &other) const
- bool **operator==** (const Studentas &other) const
- bool **operator<=** (const Studentas &other) const
- bool **operator>=** (const Studentas &other) const
- bool **operator!=** (const Studentas &other) const
- Studentas & **operator+=** (int pazymys)
- int **operator[ ]** (size_t i) const
- int & **operator[ ]** (size_t i)
- void print (std::ostream &os) const override
- const std::vector< int > & **getPaz** () const
- int **getEgz** () const
- double **getRez** () const
- size_t **getPazSkaicius** () const
- void **setEgz** (int e)
- void **setRez** (double r)
- void **addPazymys** (int p)
- void **clearPazymiai** ()
- double **vidurkis** () const
- double **mediana** () const
- void **skaiciuotiRez** (int tipas)
- bool **islaike** () const

Public Member Functions inherited from Zmogus

- **Zmogus** (const std::string &vardas, const std::string &pavarde)
- **Zmogus** (const Zmogus &other)
- **Zmogus** (Zmogus &&other) noexcept
- Zmogus & **operator=** (const Zmogus &other)
- Zmogus & **operator=** (Zmogus &&other) noexcept
- const std::string & **getVardas** () const
- const std::string & **getPavarde** () const
- void **setVardas** (const std::string &v)
- void **setPavarde** (const std::string &p)

Friends

- std::ostream & **operator**<< (std::ostream &os, const Studentas &s)
- std::istream & **operator**>> (std::istream &is, Studentas &s)

Additional Inherited Members

Protected Attributes inherited from Zmogus

- std::string **vardas_**
- std::string **pavarde_**

4.1.1 Member Function Documentation

4.1.1.1 print()

```
void Studentas::print (
            std::ostream & os) const  [override], [virtual]
```

Implements Zmogus.

The documentation for this class was generated from the following files:

- studentas.h
- studentas.cpp

4.2 TestRow Struct Reference

Public Attributes

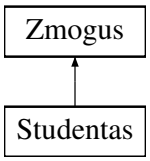
- std::string **konteineris**
- std::string **strategija**
- std::string **failas**
- int **studentu_sk** = 0
- double **skaitymas** = 0.0
- double **rusiavimas** = 0.0
- double **skirstymas** = 0.0
- double **bendras** = 0.0

The documentation for this struct was generated from the following file:

- testavimas.cpp

4.3 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- **Zmogus** (const std::string &vardas, const std::string &pavarde)
- **Zmogus** (const Zmogus &other)
- **Zmogus** (Zmogus &&other) noexcept
- Zmogus & **operator=** (const Zmogus &other)
- Zmogus & **operator=** (Zmogus &&other) noexcept
- const std::string & **getVardas** () const
- const std::string & **getPavarde** () const
- void **setVardas** (const std::string &v)
- void **setPavarde** (const std::string &p)
- virtual void **print** (std::ostream &os) const =0

Protected Attributes

- std::string **vardas_**
- std::string **pavarde_**

The documentation for this class was generated from the following files:

- zmogus.h
- zmogus.cpp

Chapter 5

File Documentation

5.1 deque_ops.h

```
00001 #ifndef DEQUE_OPS_H
00002 #define DEQUE_OPS_H
00003
00004 #include "studentas.h"
00005 #include <deque>
00006 #include <string>
00007
00008 void skaitymas_is_failo_d(const std::string& filename, std::deque<Studentas>& studentai);
00009 void isvedimas_i_faila_d(const std::deque<Studentas>& studentai,
00010     const std::string& filename,
00011     const std::string& kategorija);
00012 void pasirinkimo_metodas_d(int tipas, std::deque<Studentas>& studentai);
00013 void rusiavimas_d(std::deque<Studentas>& studentai);
00014
00015 // Originali strategija
00016 double skirstymas_i_grupes_d(const std::deque<Studentas>& visi,
00017     std::deque<Studentas>& kieti,
00018     std::deque<Studentas>& vargsai);
00019
00020 // STRATEGIJA 1 du nauji konteineriai, originalas lieka nepakeistas
00021 double skirstymas_s1_d(const std::deque<Studentas>& visi,
00022     std::deque<Studentas>& kieti,
00023     std::deque<Studentas>& vargsai);
00024
00025 // STRATEGIJA 2 vienas naujas konteineris + erase/remove if
00026 double skirstymas_s2_d(std::deque<Studentas>& studentai,
00027     std::deque<Studentas>& vargsai);
00028
00029 // STRATEGIJA 3 std::partition
00030 double skirstymas_s3_d(std::deque<Studentas>& studentai,
00031     std::deque<Studentas>& vargsai);
00032
00033 #endif
```

5.2 list_ops.h

```
00001 #ifndef LIST_OPS_H
00002 #define LIST_OPS_H
00003
00004 #include "studentas.h"
00005 #include <list>
00006 #include <string>
00007
00008 void skaitymas_is_failo_l(const std::string& filename, std::list<Studentas>& studentai);
00009 void isvedimas_i_faila_l(const std::list<Studentas>& studentai,
00010     const std::string& filename,
00011     const std::string& kategorija);
00012 void pasirinkimo_metodas_l(int tipas, std::list<Studentas>& studentai);
00013 void rusiavimas_l(std::list<Studentas>& studentai);
00014
00015 // Originali strategija
00016 double skirstymas_i_grupes_l(const std::list<Studentas>& visi,
00017     std::list<Studentas>& kieti,
```

```
00018     std::list<Studentas>& vargsai);
00019
00020 // STRATEGIJA 1 du nauji konteineriai, originalas lieka nepakeistas
00021 double skirstymas_s1_l(const std::list<Studentas>& visi,
00022     std::list<Studentas>& kieti,
00023     std::list<Studentas>& vargsai);
00024
00025 // STRATEGIJA 2 vienas naujas konteineris + erase/remove if
00026 double skirstymas_s2_l(std::list<Studentas>& studentai,
00027     std::list<Studentas>& vargsai);
00028
00029 // STRATEGIJA 3 splice - nulines kopijos
00030 double skirstymas_s3_l(std::list<Studentas>& studentai,
00031     std::list<Studentas>& vargsai);
00032
00033 #endif
```

5.3 studentas.h

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "zmogus.h"
00005 #include <vector>
00006 #include <iostream>
00007 #include <iomanip>
00008 #include <sstream>
00009
00010 //
00011 // Isvestine klase Studentas : public Zmogus
00012 //
00013 // Paveldi is abstrakcios klases Zmogus:
00014 //   - vardas_, pavarde_ laukus
00015 //   - getVardas(), getPavarde(), setVardas(), setPavarde()
00016 //   - Rule of Five is bazines klases
00017 //
00018 // Papildo savo laukais: paz_, egz_, rez_
00019 // Realizuoja gryna virtualu metoda: print()
00020 //
00021
00022 class Studentas : public Zmogus {
00023 private:
00024     std::vector<int> paz_;
00025     int egz_;
00026     double rez_;
00027
00028 public:
00029     // Konstruktoriai
00030
00031     // Numatytasis konstruktorius
00032     Studentas();
00033
00034     // Parametrinis konstruktorius
00035     Studentas(const std::string& vardas,
00036         const std::string& pavarde,
00037         const std::vector<int>& paz,
00038         int egz);
00039
00040     // Kopijavimo konstruktorius
00041     Studentas(const Studentas& other);
00042
00043     // Perkelimo konstruktorius
00044     Studentas(Studentas&& other) noexcept;
00045
00046     // Destruktorius
00047     ~Studentas();
00048
00049     //  Priskyrimo operatoriai
00050     Studentas& operator=(const Studentas& other);
00051     Studentas& operator=(Studentas&& other) noexcept;
00052
00053     //  Palyginimo operatoriai
00054     bool operator<(const Studentas& other) const;
00055     bool operator>(const Studentas& other) const;
00056     bool operator==(const Studentas& other) const;
00057     bool operator<=(const Studentas& other) const;
00058     bool operator>=(const Studentas& other) const;
00059     bool operator!=(const Studentas& other) const;
00060
00061     // Prideti pazymi: s += 8
00062     Studentas& operator+=(int pazymys);
00063
00064     // Prieiga prie paz_[i]
```

```
00065     int  operator[](size_t i) const;
00066     int& operator[](size_t i);
00067
00068     // SRAUTU OPERATORIAI
00069
00070     // Isveda: Vardas  Pavarde  [paz1 ... pazN]  Egz: E  Rez: R
00071     friend std::ostream& operator<<(std::ostream& os, const Studentas& s);
00072
00073     // Nuskaito eilute: Vardas Pavarde paz1 paz2 ... pazN egz
00074     friend std::istream& operator>>(std::istream& is, Studentas& s);
00075
00076     //
00077     // Virtualus metodas  realizuoja bazines klases gryna virtualu metoda
00078     //
00079
00080     void print(std::ostream& os) const override;
00081
00082     //
00083     // GETTERIAI
00084     //
00085
00086     const std::vector<int>& getPaz()         const { return paz_; }
00087     int                     getEgz()         const { return egz_; }
00088     double                  getRez()         const { return rez_; }
00089     size_t                  getPazSkaicius() const { return paz_.size(); }
00090
00091     //
00092     // SETTERIAI
00093     //
00094
00095     void setEgz(int e) { egz_ = e; }
00096     void setRez(double r) { rez_ = r; }
00097
00098     //
00099     // METODAI
00100     //
00101
00102     void   addPazymys(int p);
00103     void   clearPazymiai();
00104     double vidurkis() const;
00105     double mediana()  const;
00106     void   skaiciuotiRez(int tipas); // 1=vidurkis, 2=mediana
00107     bool   islaike() const { return rez_ >= 5.0; }
00108 };
00109
00110 #endif
```

5.4 utils.h

```
00001 #ifndef UTILS_H
00002 #define UTILS_H
00003
00004 #include <string>
00005
00006 std::string ivesti_varda_ar_pavarde(const std::string& zinute);
00007 int ivesties_tikrinimas(const std::string& zinute);
00008 int ivesties_skaicius(const std::string& zinute);
00009 int skaiciavimo_metodas();
00010
00011 #endif
```

5.5 vector_ops.h

```
00001 #ifndef VECTOR_OPS_H
00002 #define VECTOR_OPS_H
00003
00004 #include "studentas.h"
00005 #include <vector>
00006 #include <string>
00007
00008 // Skaitymas ir isvedimas
00009 void skaitymas_is_failo(const std::string& filename, std::vector<Studentas>& studentai);
00010 void isvedimas(const std::vector<Studentas>& studentai, int metodas);
00011 void isvedimas_i_faila(const std::vector<Studentas>& studentai,
00012     const std::string& filename,
00013     const std::string& kategorija);
00014
00015 // Skaiciavimai
```

```
00016 void pasirinkimo_metodas(int tipas, std::vector<Studentas>& studentai);
00017
00018 // rusiavimas
00019 void rusiavimas(std::vector<Studentas>& studentai, int budas);
00020 int  pasirinkimas_rusiavimo_budo();
00021
00022 // skirtstymas i grupes
00023 void   skirstymas_i_grupes(const std::vector<Studentas>& visi,
00024     std::vector<Studentas>& kieti,
00025     std::vector<Studentas>& vargsai);
00026
00027 double skirstymas_s1(const std::vector<Studentas>& studentai,
00028     std::vector<Studentas>& kieti,
00029     std::vector<Studentas>& vargsai);
00030
00031 double skirstymas_s2(std::vector<Studentas>& studentai,
00032     std::vector<Studentas>& vargsai);
00033
00034 double skirstymas_s3(std::vector<Studentas>& studentai,
00035     std::vector<Studentas>& vargsai);
00036
00037 #endif
```

5.6 zmogus.h

```
00001 #pragma once
00002 #ifndef ZMOGUS_H
00003 #define ZMOGUS_H
00004
00005 #include <string>
00006 #include <iostream>
00007
00008 //
00009 // Abstrakti bazine klase Zmogus
00010 //
00011 // Negalima sukurti Zmogus tipo objektu tiesiogiai
00012 // klase abstrakti del grynai virtualaus destruktoriaus.
00013 // Is jos galima kurti tik isvesdines klases (pvz. Studentas).
00014 //
00015
00016 class Zmogus {
00017 protected:
00018     std::string vardas_;
00019     std::string pavarde_;
00020
00021 public:
00022     // --- Konstruktoriai (Rule of Five) ---
00023
00024     Zmogus();
00025     Zmogus(const std::string& vardas, const std::string& pavarde);
00026     Zmogus(const Zmogus& other);
00027     Zmogus(Zmogus&& other) noexcept;
00028
00029     // Grynas virtualus destruktorius  daro klase abstrakcia
00030     // Isvestines klases privalo override'inti
00031     virtual ~Zmogus() = 0;
00032
00033     // --- Priskyrimo operatoriai (Rule of Five) ---
00034
00035     Zmogus& operator=(const Zmogus& other);
00036     Zmogus& operator=(Zmogus&& other) noexcept;
00037
00038     // --- Getteriai ---
00039
00040     const std::string& getVardas()  const { return vardas_; }
00041     const std::string& getPavarde() const { return pavarde_; }
00042
00043     // --- Setteriai ---
00044
00045     void setVardas(const std::string& v) { vardas_ = v; }
00046     void setPavarde(const std::string& p) { pavarde_ = p; }
00047
00048     // --- Grynas virtualus metodas  isvestines klases privalo realizuoti ---
00049
00050     // Isveda informacija apie objekta i srautua
00051     virtual void print(std::ostream& os) const = 0;
00052 };
00053
00054 // Globalus isvedimo operatorius naudoja virtual print()
00055 inline std::ostream& operator<<(std::ostream& os, const Zmogus& z) {
00056     z.print(os);
00057     return os;
00058 }
```

```
00059
00060 #endif // ZMOGUS_H
```

Index